

Core Components of Simulated Annealing

Energy Function (Cost/Objective Function)

The **Energy Function** (often called the **Cost Function** or **Objective Function**) represents the quality or fitness of a solution.

- **Definition:**
 - The energy function assigns a value (or "cost") to each possible solution in the search space.
 - Lower energy values represent better solutions, and higher energy values represent worse solutions.
- **Role in SA:**
 - SA uses the energy function to evaluate how good or bad a solution is. The algorithm attempts to minimize this function, just as a physical material seeks to minimize its energy state during annealing.
- **Example:**
 - In the Travelling Salesman Problem (TSP), the energy function could be the total distance traveled. The goal would be to minimize this distance.

Temperature (T)

The **Temperature** (T) controls the algorithm's willingness to accept worse solutions.

- **Initial High Temperature:**

- At the start of the algorithm, the temperature is high, which means that the system can explore freely by accepting both better and worse solutions.
- The probability of accepting a worse solution is determined by the equation:

$$P(\Delta E) = \exp \left(\frac{-\Delta E}{T} \right)$$

where ΔE is the difference between the current solution and the new solution.

- If ΔE is positive (i.e., the new solution is worse), there is still a chance that the solution will be accepted, especially when T is high.
- **Decreasing Temperature:**
 - As the temperature decreases, the algorithm becomes less likely to accept worse solutions. Eventually, when the temperature is near zero, the system only accepts better solutions.
- **Intuition:**
 - At high temperatures, the algorithm explores more of the solution space (including suboptimal solutions).
 - At low temperatures, the algorithm focuses on refining the best solution found so far.

Cooling Schedule

The **Cooling Schedule** determines how the temperature is decreased over time.

- **Cooling Function:**

- The cooling schedule specifies the rate at which the temperature decreases as the algorithm progresses.
- Common cooling schedules include:

- **Linear Cooling:** $T = T_0 - \alpha k$, where k is the iteration number.

- **Exponential Cooling:** $T = T_0 \cdot \alpha^k$, where $\alpha < 1$ is a constant.

- **Logarithmic Cooling:** $T = \frac{T_0}{\log(1+k)}$, where k is the iteration number.

- **Choosing the Right Cooling Schedule:**

- A **slow cooling schedule** provides more time for the algorithm to explore, increasing the chances of finding a global optimum but at the cost of longer runtime.
 - A **fast cooling schedule** may lead to quicker convergence, but the system might miss the global optimum and get stuck in a local minimum.

The Simulated Annealing Algorithm

1. Initialization Phase

- **Initial Solution:**
 - The algorithm begins with a randomly selected initial solution from the solution space. This solution is often generated arbitrarily, but it should belong to the set of all possible solutions for the given optimization problem.
 - Example: In the Travelling Salesman Problem (TSP), an initial solution might be a random order of visiting cities.
- **Initial Temperature (T_0):**
 - The algorithm sets an initial temperature (T_0), typically high, to allow for broad exploration of the solution space. The high temperature enables the algorithm to potentially accept worse solutions early on.
 - The initial temperature is a key hyperparameter that influences the initial exploration phase.

2. Generate a New Solution (Neighbor Selection)

- **Neighboring Solution:**
 - From the current solution, a new neighboring solution is generated by making a small, random perturbation or modification to the current solution.
 - The neighboring solution should be similar to the current one, ensuring that the search progresses gradually rather than making large jumps in the solution space.
 - Example: In the TSP, a neighboring solution could be generated by swapping the order of two cities in the current route.

3. Evaluate the New Solution

- **Energy Function (Objective Function):**
 - The quality of both the current and new solutions is evaluated using the **energy function** (also known as the **cost** or **objective function**).
 - The energy function represents the quality of the solution—lower values represent better solutions.
- **Energy Difference (ΔE):**
 - Compute the energy difference between the current solution and the new (neighboring) solution:

$$\Delta E = E_{\text{new}} - E_{\text{current}}$$

- **Interpretation of ΔE :**
 - If $\Delta E < 0$, the new solution is better (lower cost) than the current solution.
 - If $\Delta E > 0$, the new solution is worse (higher cost) than the current solution.

4. Acceptance Criteria

- **Better Solutions ($\Delta E < 0$):**
 - If the new solution has a lower cost (i.e., $\Delta E < 0$), it is immediately accepted as the current solution.
 - This ensures that the algorithm always moves toward improving solutions when they are found.
- **Worse Solutions ($\Delta E > 0$):**
 - If the new solution is worse than the current solution (i.e., $\Delta E > 0$), it can still be accepted based on a probability:

$$P(\Delta E) = \exp\left(\frac{-\Delta E}{T}\right)$$

- **Explanation:**
 - The probability of accepting a worse solution depends on both the temperature (T) and the energy difference (ΔE).
 - At high temperatures, this probability is higher, allowing the algorithm to explore the solution space freely. At low temperatures, the probability of accepting worse solutions decreases, making the algorithm more selective.
- This mechanism allows SA to escape local optima by occasionally accepting worse solutions, especially in the early stages when the temperature is high.

5. Update Temperature

- **Cooling Schedule:**

- After evaluating and accepting (or rejecting) the new solution, the temperature is reduced according to a predefined **cooling schedule**. The cooling schedule dictates how quickly or slowly the temperature decreases over time.

Types of Cooling Schedules:

1. **Linear Cooling:**

$$T = T_0 - \alpha \cdot k$$

- α is a constant cooling rate.
- The temperature decreases linearly with each iteration.
- Suitable for problems where a steady decrease in temperature is desired.

2. **Exponential Cooling:**

$$T = T_0 \cdot \alpha^k$$

- α is a constant between 0 and 1.
- The temperature decreases exponentially with each iteration.
- This cooling schedule allows for faster cooling in the later stages of the algorithm.

3. Logarithmic Cooling:

$$T = \frac{T_0}{\log(1 + k)}$$

- The temperature decreases more slowly, especially in the early iterations, allowing for thorough exploration of the solution space before converging.
 - Logarithmic cooling ensures a more controlled, gradual reduction in temperature and is more computationally expensive.
-
- **Selection of Cooling Schedule:**
 - The choice of cooling schedule depends on the nature of the problem. Faster cooling schedules (e.g., exponential) might be used when quick convergence is desired, while slower cooling schedules (e.g., logarithmic) might be preferred for more thorough exploration.

6. Termination Criteria

- **Freezing the System:**

- The process continues until the system is "frozen," which typically means the temperature approaches zero (or becomes very low), effectively stopping the acceptance of worse solutions. At this point, the algorithm is highly selective and only accepts better solutions.
- Another common termination criterion is to stop the process when no significant improvement is observed over a number of iterations.

- **Convergence:**

- The algorithm is said to converge when no further improvements can be made, or the temperature has decreased to a point where the solution has stabilized.
- The solution at this point is either the global optimum or a near-optimal solution.

- **Stopping Conditions:**

- The algorithm can also terminate based on:
 - A maximum number of iterations.
 - A predefined time limit.
 - The difference between consecutive solutions becoming negligible (indicating convergence).

The Simulated Annealing algorithm

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state

current $\leftarrow$ *problem*.INITIAL

for $t = 1$ **to** ∞ **do**

$T \leftarrow$ *schedule*(t)

if $T = 0$ **then return** *current*

next $\leftarrow$ a randomly selected successor of *current*

$\Delta E \leftarrow$ VALUE(*current*) – VALUE(*next*)

if $\Delta E > 0$ **then** *current* $\leftarrow$ *next*

else *current* $\leftarrow$ *next* only with probability $e^{-\Delta E/T}$

Applications of SA

- In addition to the well-known applications like the **Travelling Salesman Problem (TSP)**, **job scheduling**, **neural network training**, and **VLSI design**, Simulated Annealing (SA) has a wide variety of other applications in artificial intelligence (AI) and related fields. In addition to the well-known applications like the **Travelling Salesman Problem (TSP)**, **job scheduling**, **neural network training**, and **VLSI design**, Simulated Annealing (SA) has a wide variety of other applications in artificial intelligence (AI) and related fields. Below are additional applications where SA has been effectively used:
- **Feature Selection in Machine Learning:**
- **Hyperparameter Tuning in Machine Learning**
- **Reinforcement Learning Policy Optimization**
- **AI for Game Playing (Game Theory)**
- **Clustering in Data Mining**
- **Path Planning in Robotics**
- **Genetic Algorithm Hybridization**
- **Computer Vision: Image Segmentation**
- **Protein Folding in Bioinformatics**
- **Autonomous Vehicle Navigation**
- **AI for Financial Portfolio Optimization**
- **Spatial Layout Design (Architectural Optimization)**
- **Circuit Routing in Electronic Design Automation (EDA)**
- **3D Object Recognition**

Why Simulated Annealing Works in These Applications

Flexibility in Handling Non-Convex and Complex Objective Functions:

- Many real-world problems, especially those in AI, have non-convex objective functions with multiple local optima. SA is well-suited for such problems because it doesn't just move to better solutions but also explores worse solutions, helping to avoid local optima.

Works Well in Large, Combinatorial Search Spaces:

- In problems like TSP, job scheduling, and VLSI design, the solution space is vast and highly combinatorial. SA's ability to explore large portions of the search space by accepting suboptimal moves early in the process helps it find global or near-global optima efficiently.

Limitations and Challenges of Simulated Annealing

- 1. Slow Convergence**
- 2. Sensitivity to Cooling Schedule**
- 3. Not Ideal for Small-Scale Problems**
- 4. Difficulty in Parameter Tuning**

Potential Solutions to These Challenges

- **Hybrid Algorithms:**
 - Combine SA with other optimization techniques (e.g., **genetic algorithms** or **tabu search**) to take advantage of SA's exploration capabilities while using faster algorithms for local exploitation.
- **Adaptive Cooling Schedules:**
 - Implement **adaptive cooling schedules** that dynamically adjust the temperature based on the performance of the algorithm. For example, if improvements are happening rapidly, the schedule can slow down the cooling rate.
- **Parallelization:**
 - One way to speed up SA and improve exploration is to use **parallelization**. Multiple instances of the algorithm can explore different parts of the solution space simultaneously, helping to identify promising areas more quickly.

N Queen Formulation as Local Search

Problem Definition (State, Objective, and Cost Function):

1. **State:**
 - A state in the N-Queens problem is defined by the positions of N queens on an $N \times N$ board, where each queen is in a unique column (a common constraint to simplify the problem).
 - For each column, we store the row position of the queen.
2. **Objective Function:**
 - The objective is to minimize the number of conflicts between queens. A conflict is defined as two queens attacking each other (sharing the same row or diagonal).
3. The **cost function** $C(S)$ of a state S is the total number of pairs of queens that are in conflict.
4. **Initial State:**
 - You start with a random initial configuration, where each queen is placed in a random row within its column. This configuration may have many conflicts initially.
5. **Neighboring States (Set of Actions):**
 - A neighboring state is generated by moving a single queen to a different row within the same column. The set of possible actions is thus defined as moving any queen from its current row to any other row in the same column.
6. **Cost Function:**
 - The cost function $C(S)$ calculates the number of queens attacking each other. The lower the cost, the better the state.

N Queens- Hill Climbing

Greedy Hill Climbing: Possible Set of Actions for Greedy: For each queen, consider moving it to a different row in its column. The first move that reduces conflicts will be chosen.

Example:

- Assume you have a queen in column 1, row 3.
- Check each possible row (1, 2, 4, 5, etc.) in column 1. As soon as you find a row that reduces the total number of conflicts (e.g., moving the queen to row 2 reduces conflicts), make that move and stop further exploration.
- This approach **stops at the first improvement**.

Distinctive Feature: It **doesn't explore all possible moves**. It just makes the **first move that leads to a better configuration**, without further comparisons.

Steepest-Ascent Hill Climbing: Possible Set of Actions for Steepest-Ascent: For each queen, consider all possible row moves in its column. Compare all possible moves and choose the one that gives the largest reduction in conflicts.

Example:

- Assume you have a queen in column 2, row 5.
- Evaluate every possible move of every queen across all columns (e.g., column 1, column 2, etc.) to see which move results in the maximum reduction in conflicts.
- If moving the queen in column 3 from row 4 to row 1 results in the **largest decrease in conflicts**, you select this move.

Distinctive Feature: The algorithm looks for the **best overall move** across the entire board, rather than stopping at the first improvement. It evaluates the entire board before deciding on the best move.

First-Choice Hill Climbing: Possible Set of Actions for First-Choice: Randomly select a move (new row for a queen within its column). If it improves the solution (i.e., reduces conflicts), make the move.

Example:

- Start with any random initial configuration.
- Randomly choose a queen (say the one in column 4) and randomly move it to a different row (row 1, row 2, etc.).
- If this random move reduces conflicts, make the move immediately and continue the process.

Distinctive Feature: Moves are **randomly selected** without evaluating all possible neighbors. The algorithm makes the first move it finds that improves the configuration.

Stochastic Hill Climbing: Possible Set of Actions for Stochastic Hill Climbing: For each queen, randomly choose a neighboring position but with **higher probability for moves that reduce conflicts more**.

Example:

- Assume you have a queen in column 3, row 4.
- Evaluate all possible moves in its column (rows 1, 2, 3, 5, etc.), assigning higher probabilities to moves that reduce conflicts more. For example:
 - Moving the queen to row 1 reduces conflicts by 3 (higher probability of being chosen).
 - Moving the queen to row 2 reduces conflicts by 1 (lower probability).
 - Moving the queen to row 3 increases conflicts (very low probability).
- The algorithm **randomly selects a move based on these probabilities**.

Distinctive Feature: Moves are selected **probabilistically**, with better moves having higher chances, but **there's still a possibility of choosing a suboptimal move**.

Random-Restart Hill Climbing: Possible Set of Actions for Random-Restart Hill Climbing: Randomly restart the hill climbing process with a completely new random board configuration when a local minimum is reached.

Example:

- Start with a random configuration and apply **steepest-ascent hill climbing**.
- If you reach a local minimum (a state where no further improvements can be made), **restart** with a completely new random configuration.
- Repeat the process until you find a solution with 0 conflicts.

Distinctive Feature: The algorithm **restarts from scratch** when stuck in a local minimum, avoiding the possibility of staying stuck in a suboptimal configuration.

N Queens: Simulated Annealing

1. Generating Neighboring States: A neighboring state is generated by moving any one queen to a different row in the same column.

2. Move Acceptance Based on Temperature:

- **High Temperature:** At the start, the algorithm makes more random moves, even if the new state has more conflicts. The algorithm is willing to explore the search space more broadly and escape local minima.
 - **Action:** Move a randomly selected queen to any other row in its column, even if it results in more conflicts (higher cost).
- **Low Temperature:** As the temperature decreases, the algorithm becomes less likely to accept bad moves. It behaves more like steepest-ascent hill climbing, focusing on improving moves (those that reduce conflicts).
 - **Action:** Select a queen and move it to a position that either reduces conflicts or keeps the number of conflicts the same. However, if a small cost increase happens, it will still be accepted with some probability.

3. Cooling Schedule and Behavior:

- **Cooling Schedule:** The temperature decreases gradually, controlling how likely the algorithm is to accept worse moves. At high temperatures, the algorithm is more exploratory, while at low temperatures, it becomes more exploitative, fine-tuning the solution.
- **Actions based on Temperature:**
 - **High Temperature:**
 - Explore random neighboring states, accepting worse states frequently.
 - This prevents the algorithm from getting stuck in a local minimum early on.
 - **Low Temperature:**
 - The algorithm becomes more selective, accepting only moves that improve the current solution or slightly worsen it.

Example Walkthrough of Simulated Annealing for N-Queens:

Step 1: Initialize

- Start with a random configuration:
 - Example: $N = 8$, queens placed randomly in each column:
 - State: $[4, 6, 3, 7, 1, 8, 5, 2]$ (where the numbers represent the row of the queen in each column).
 - Initial cost: 5 conflicts (e.g., some queens share the same diagonal).

Step 2: Choose a Neighboring State

- Pick a random queen, say the queen in column 2 (currently in row 6), and move it to a different row, say row 3. This creates a new state:
 - New state: $[4, 3, 3, 7, 1, 8, 5, 2]$.
 - New cost: 6 conflicts (more conflicts than before).

Step 3: Acceptance Decision

- At **high temperature**: The algorithm may accept this worse move with a certain probability.

Let's assume $T = 100$, and the cost increase $\Delta E = 1$:

- Acceptance probability: $P(\Delta E) = e^{-\frac{1}{100}} \approx 0.99$ (so it's very likely to accept the worse move).
- If accepted, the new state becomes the current state.

Step 4: Reduce Temperature

- Update the temperature according to the cooling schedule:
 - $T_{\text{new}} = 0.95 \times T_{\text{old}}$.

Step 5: Repeat

- Continue selecting random moves and making acceptance decisions based on the current temperature until the temperature is sufficiently low, at which point the algorithm will focus mostly on reducing the number of conflicts.

Algorithm	Optimality for Small N	Optimality for Large N	Steps for Small N	Steps for Large N
Simple Hill Climbing	70-80%	< 5%	200-300	10,000+ (fails often)
Steepest-Ascent Hill Climbing	85-90%	15-20%	150-200	8,000-12,000
First-Choice Hill Climbing	75-85%	10-15%	250-350	9,000-11,000
Stochastic Hill Climbing	80-85%	25-30%	300-400	8,000-10,000
Random-Restart Hill Climbing	95-99%	40-50%	500-600 (due to restarts)	15,000-20,000 (due to restarts)
Simulated Annealing	90-95%	70-80%	300-500	10,000-15,000

Local Beam Search

Local Beam Search is a type of heuristic search algorithm that explores the state space in search of a solution by maintaining multiple states (or solutions) at each step rather than just one. Unlike algorithms like hill climbing or gradient descent that work with a single state and progress iteratively, local beam search keeps track of **k** candidate states at each step, aiming to improve efficiency by exploring multiple parts of the search space simultaneously.

This method is effective in avoiding the problem of getting stuck in local optima, which is common in many single-state local search methods. It does so by allowing parallel exploration of the search space.

Core Concepts of Local Beam Search

1. **Multiple States:**
 - Instead of one state, local beam search operates with **k** states at a time. This means that at each iteration, there is a parallel exploration happening in **k** different regions of the state space.
2. **Successor Generation:**
 - For each of the **k** states, their successors (neighboring states) are generated by applying actions or transformations. These are essentially new states that are one step away from the current states.
3. **Selection of Best States:**
 - From all the successors generated across all **k** states, the best **k** states are selected based on some evaluation criteria (e.g., the value of an objective function). These **k** states then become the current states for the next iteration.
4. **Termination:**
 - The process continues until either a goal state (a solution) is found, or a termination condition is met (such as a maximum number of iterations, time limit, or convergence).

Algorithm Steps

1. **Initialization:**
 - Start with k randomly chosen initial states (or heuristically selected states if prior information is available). These states form the first beam.
2. **Generate Successors:**
 - For each state in the current beam, generate all possible successors. Successors are neighboring states that can be reached by applying some action or transformation to the current state.
3. **Evaluate Successors:**
 - Evaluate the successors using an evaluation function or objective function that measures the quality of the state (e.g., distance to goal, cost, or fitness score).
4. **Select the Best k States:**
 - From the pool of all successors generated across the k states, select the top k states based on the evaluation function. These selected states will form the next beam.
5. **Repeat:**
 - Repeat the process of generating successors and selecting the best states until a goal state is found or a stopping criterion is met (such as a limit on the number of iterations).
6. **Termination:**
 - The algorithm terminates if a solution (goal state) is found or if it reaches a stopping criterion such as a time limit, or if there are no improvements in the best states after several iterations.

Example: Traveling Salesman Problem (TSP)

Consider the **Traveling Salesman Problem (TSP)**, where the objective is to find the shortest possible route that visits a set of cities exactly once and returns to the starting city. Using local beam search:

1. **Initialization:**
 - Start with $k = 3$ random routes (states), each representing a different ordering of cities.
2. **Successor Generation:**
 - For each route (state), generate successors by making small changes, such as swapping two cities or reversing a part of the route.
3. **Evaluation:**
 - Calculate the total distance of each route (state) as the evaluation metric. The goal is to minimize this distance.
4. **Selection:**
 - Choose the 3 shortest routes from the pool of generated successors.
5. **Iteration:**
 - Repeat the process, generating new routes from the best 3 and selecting the shortest ones, until a satisfactory route is found or a stopping criterion is met.

Key Properties

1. **Parallel Search:**

- Unlike single-state search methods (such as hill climbing or gradient descent), local beam search performs parallel exploration of the search space. By maintaining multiple states, it can avoid local optima more effectively.

2. **Focuses on the Best:**

- Local beam search selectively keeps the best k states in each iteration. This means the search is directed toward regions of the state space that seem most promising, which makes the search more focused and efficient.

Challenges and Limitations

1. **Loss of Diversity:**

- A major challenge in local beam search is the potential **loss of diversity**. If all k states converge to a similar region of the search space, the algorithm may prematurely focus on a suboptimal region, missing better solutions elsewhere.
- **Solution: Stochastic Beam Search**, a variation of local beam search, helps mitigate this by probabilistically selecting the next k states based on their evaluation scores rather than deterministically choosing the top k . This keeps diversity in the search process.

2. **Computational Overhead:**

- Maintaining and evaluating k states in parallel can lead to higher computational costs compared to single-state search methods, especially if k is large or if generating successors is computationally expensive.

3. **Beam Width:**

- Choosing the correct number of beams (the value of k) is crucial. If k is too small, the algorithm may not explore enough of the search space. If k is too large, it could become computationally expensive without adding much benefit.

Example of Local Beam Search in N-Queens (With $k=3$)

Initial Step: Randomly Initialize 3 Board Configurations

- **State 1:** [2, 4, 6, 8, 1, 3, 5, 7]
(Cost: 6 conflicts)
- **State 2:** [3, 1, 4, 7, 2, 5, 8, 6]
(Cost: 4 conflicts)
- **State 3:** [1, 3, 5, 8, 6, 4, 2, 7]
(Cost: 5 conflicts)

Step 2: Generate Successors for Each State

For each state, generate new configurations by moving one queen to a different row in its column.

Let's generate a few successors for each state.

- **State 1:** [2, 4, 6, 8, 1, 3, 5, 7]
Possible moves for queens:
 - Move queen in column 1 (currently row 2) to rows 1, 3, ..., 8.
 - Move queen in column 2 (currently row 4) to rows 1, 2, ..., 8.
 - Continue generating successors by changing positions of other queens.
 - **Sample successor:** [1, 4, 6, 8, 1, 3, 5, 7] (Cost: 5 conflicts)

- **State 2:** [3, 1, 4, 7, 2, 5, 8, 6]

Possible moves for queens:

- Move queen in column 1 (currently row 3) to rows 1, 2, ..., 8.
- Move queen in column 2 (currently row 1) to rows 2, 3, ..., 8.
- Continue generating successors for other queens.
- **Sample successor:** [3, 2, 4, 7, 2, 5, 8, 6] (Cost: 3 conflicts)

- **State 3:** [1, 3, 5, 8, 6, 4, 2, 7]

Possible moves for queens:

- Move queen in column 1 (currently row 1) to rows 2, 3, ..., 8.
- Move queen in column 2 (currently row 3) to rows 1, 2, ..., 8.
- Continue generating successors for other queens.
- **Sample successor:** [1, 2, 5, 8, 6, 4, 2, 7] (Cost: 4 conflicts)

Step 3: Evaluate Successors and Select the Best k

After generating a large number of successors from each state, evaluate their costs (conflicts).

Here are some sample successors with their costs:

- **From State 1:**

- $[1, 4, 6, 8, 1, 3, 5, 7]$: 5 conflicts
- $[3, 4, 6, 8, 1, 3, 5, 7]$: 4 conflicts
- $[2, 1, 6, 8, 1, 3, 5, 7]$: 5 conflicts

- **From State 2:**

- $[3, 2, 4, 7, 2, 5, 8, 6]$: 3 conflicts
- $[3, 1, 4, 7, 3, 5, 8, 6]$: 4 conflicts

- **From State 3:**

- $[1, 2, 5, 8, 6, 4, 2, 7]$: 4 conflicts
- $[2, 3, 5, 8, 6, 4, 2, 7]$: 4 conflicts

Now, choose the **best 3 states** (with the lowest number of conflicts) to continue the search. For example:

- [3, 2, 4, 7, 2, 5, 8, 6] (Cost: 3 conflicts)
- [3, 4, 6, 8, 1, 3, 5, 7] (Cost: 4 conflicts)
- [1, 2, 5, 8, 6, 4, 2, 7] (Cost: 4 conflicts)

Step 4: Repeat the Process

Repeat the process by generating successors for these 3 selected states, evaluating them, and selecting the best 3 configurations to continue.

Termination:

- If one of the states reaches **0 conflicts** (no queens attacking each other), the algorithm terminates with a solution.
- If after several iterations no improvement is made, the algorithm may stop (based on a predefined stopping criterion).

Genetic Algorithm

Introduction to Evolutionary Algorithms

Evolutionary algorithms (EAs) are a class of optimization techniques inspired by the process of **natural selection** and **biological evolution**. These algorithms are particularly effective in solving complex problems where traditional methods may struggle, especially in **non-linear, multi-modal, or large-scale search spaces**.

Biological Inspiration:

- The key idea behind evolutionary algorithms is to evolve a population of candidate solutions over multiple generations. The solutions are assessed based on their "fitness," and better-performing solutions are more likely to reproduce, leading to improved generations over time.
- Just as in nature, these algorithms use **selection**, **crossover**, and **mutation** to explore and exploit the solution space.

Common Types of Evolutionary Algorithms:

- **Genetic Algorithms (GA)**: GAs mimic natural genetics and selection to search for optimal solutions through processes like crossover (recombination) and mutation. They are highly flexible and robust across different problem types.
- **Evolutionary Programming**: Focuses on evolving finite state machines or programs, often used in control systems and machine learning.
- **Genetic Programming**: An extension of GAs where the solutions are represented as computer programs (trees) and evolved to find the best-performing program for a particular task.
- **Differential Evolution**: Primarily used for continuous optimization problems, DE applies mutation based on the differences between randomly selected candidate solutions to explore the search space.

Advantages of Evolutionary Algorithms:

- **Global Search**: EAs can explore large, complex search spaces and often avoid getting stuck in local optima, unlike local search methods like gradient descent.
- **Diverse Solutions**: The population-based approach ensures diversity among solutions, which helps in covering a broader search space and escaping local traps.
- **Parallelism**: Evolutionary algorithms naturally support parallelism since multiple candidate solutions can be evaluated and evolved simultaneously.

Why Genetic Algorithms?

Among the different evolutionary algorithms, **Genetic Algorithms (GA)** stand out due to their heuristic-based approach, which makes them **robust**, **flexible**, and applicable across a wide variety of optimization problems.

Key Features of Genetic Algorithms:

1. **Population-Based Search:** Unlike deterministic algorithms (e.g., greedy or local search), GAs work with a population of solutions rather than a single candidate. This increases the chance of exploring diverse areas of the search space, leading to more global solutions.
2. **Stochastic Nature:** GAs employ stochastic operators (e.g., crossover and mutation) that introduce randomness, allowing them to explore areas of the search space that might be overlooked by deterministic methods.
3. **Adaptability:** GAs can be applied to both **discrete** (e.g., job scheduling) and **continuous** (e.g., minimizing cost functions) optimization problems. They are especially well-suited to problems where the search space is large, complex, or poorly understood.
4. **Handling Large Search Spaces:** GAs are effective when the problem has a large number of possible solutions (combinatorial explosion) or a non-linear and multi-modal landscape. For example, GAs can handle problems where the number of potential solutions grows exponentially with the size of the input, such as the traveling salesman problem (TSP).

Why Use GAs Over Traditional Methods?

- **Not Dependent on Gradient Information:** Unlike algorithms like gradient descent, GAs do not require derivatives or gradient information, making them useful for non-differentiable, discontinuous, or noisy objective functions.
- **Global Optimization:** GAs are often used in cases where global optimization is desired, as they are less likely to get trapped in local optima compared to traditional optimization methods.
- **Robust in Multi-Objective Problems:** GAs can handle problems with multiple conflicting objectives by maintaining a diverse set of solutions, potentially finding trade-offs (Pareto optimality).

Biological Inspiration Behind Genetic Algorithms

1. Natural Evolution

Natural evolution is driven by a set of core biological processes, namely **selection**, **reproduction**, **mutation**, and **survival of the fittest**. These processes ensure that over generations, species become better adapted to their environment.

Charles Darwin's Theory of Evolution:

- In the mid-19th century, **Charles Darwin** proposed the theory of **natural selection**. According to this theory, individuals in a population with traits best suited to their environment have a higher probability of surviving and reproducing. Over time, these advantageous traits become more common in the population, leading to the gradual evolution of the species.
- **"Survival of the fittest"**: This concept explains that the fittest individuals (those with traits that provide a survival or reproductive advantage) are more likely to survive and pass on their genetic material. Over generations, the population evolves, with fitter individuals dominating the gene pool.

Key Biological Concepts:

- **Selection**: Nature "selects" individuals that are better suited to their environment. These individuals are more likely to reproduce, passing their genes on to the next generation.
- **Reproduction**: Genetic material from two parent organisms is combined to create offspring, which inherit a mix of their parents' traits.
- **Mutation**: Occasionally, random changes (mutations) occur in an organism's genetic material. These mutations can introduce new traits that may offer advantages or disadvantages for survival.
- **Population**: A group of individuals of the same species living and interacting in the same environment.

How Genetic Algorithms Mimic Nature

Genetic Algorithms (GAs) simulate the process of natural evolution to search for optimal or near-optimal solutions to complex problems. Here's how GAs mimic the biological processes of evolution:

Chromosomes, Genes, and Populations in GAs:

- **Chromosomes (Solutions):** In GAs, a **chromosome** represents a potential solution to the optimization problem. Just like in biology, chromosomes consist of **genes**, where each gene represents a variable or decision within the problem domain.
 - Example: For a scheduling problem, a chromosome might represent a complete schedule, and each gene would represent a specific task's time slot.
- **Genes (Variables):** In biological systems, genes are units of inheritance. In GAs, **genes** are the individual elements that make up a solution (chromosome). Each gene typically represents a value or decision variable in the problem being solved.
 - Example: In a traveling salesman problem (TSP), each gene could represent a specific city in the route.
- **Population (Set of Solutions):** A **population** in GAs is a collection of chromosomes (candidate solutions). This population evolves over successive generations, mimicking the process of evolution.
 - The size of the population can affect how thoroughly the search space is explored: a larger population increases diversity but comes with a higher computational cost.

Fitness Function:

- Just like in nature, where organisms with beneficial traits are more likely to survive, in GAs, each solution (chromosome) is evaluated based on how well it solves the optimization problem. This is done through a **fitness function**, which assigns a score to each chromosome based on its performance relative to the problem's objective.
 - Example: In a minimization problem, such as reducing production costs, the fitness function would evaluate how close each candidate solution is to the optimal (lowest) cost.

Evolutionary Processes in GAs:

1. **Selection:**

- In GAs, selection is analogous to natural selection, where better solutions (those with higher fitness scores) are more likely to be selected to reproduce and pass their genetic information to the next generation. Various selection methods, such as **roulette wheel selection** or **tournament selection**, are used to choose parents based on fitness.
- The idea is that fitter solutions are more likely to be selected, but weaker solutions still have a chance, maintaining diversity in the population.

2. **Crossover (Recombination):**

- Just as biological reproduction mixes genetic material from two parents, in GAs, crossover combines the genes of two parent solutions to create new offspring. The offspring inherit traits (genes) from both parents, which allows the algorithm to explore new areas of the solution space.
- Example: In a simple crossover operation, two parent chromosomes are split at a random point, and segments are swapped to create two new offspring.

3. **Mutation:**

- Mutation in GAs is similar to biological mutation, introducing small random changes in the genes of offspring to maintain diversity and avoid premature convergence (getting stuck in local optima). These small perturbations ensure that the algorithm doesn't become too focused on a small part of the search space and helps explore new areas.
- Example: In binary-encoded GAs, a mutation might flip a bit from 0 to 1 or vice versa.

4. **Survival of the Fittest:**

- After selection, crossover, and mutation, the population is evaluated again, and the best-performing individuals (chromosomes) are carried over to the next generation. This process is repeated over multiple generations until a termination condition (e.g., a satisfactory solution or a maximum number of generations) is met.

Genetic Algorithm Flowchart

1. **Start:** Initialize the GA process.
2. **Initialize Population:**
 - Generate an initial population of random candidate solutions.
3. **Evaluate Fitness:**
 - Use the fitness function to evaluate each candidate solution in the population. Assign a fitness score based on how well it solves the problem.
4. **Selection:**
 - Select pairs of parent solutions based on their fitness scores. Fitter individuals are more likely to be chosen for reproduction.
5. **Crossover:**
 - Perform crossover on selected parents to generate offspring. The genetic material from both parents is recombined to create new solutions.
6. **Mutation:**
 - Apply mutation to the offspring with a certain probability to introduce randomness and maintain diversity in the population.
7. **Next Generation:**
 - Replace the old population with the new one (offspring). Evaluate the fitness of the new population.
8. **Terminate or Repeat:**
 - Check if the termination condition is met (e.g., maximum generations, fitness threshold). If not, repeat the process starting from **Selection**.
 - If the termination condition is met, the algorithm stops, and the best solution is returned.

Fitness Function and Evaluation

Role of the Fitness Function

- The fitness function evaluates how well each solution (chromosome) solves the problem.
- **Maximization vs. Minimization:**
 - Depending on the problem, the fitness function can be designed to either **maximize** or **minimize** an objective.
 - Example: In a cost minimization problem, lower fitness values are better, while in profit maximization, higher fitness values are preferred.

Designing the Fitness Function

- Must be aligned with the optimization problem's objectives to guide the search effectively.
- **Fitness Scaling:**
 - Methods like **rank-based scaling** or **proportional fitness** prevent a single solution from dominating the population early on, maintaining a balanced selection process.
 - Example: In scaling, solutions are ranked and normalized to ensure a fair probability of being selected for reproduction, regardless of their raw fitness values.

Selection Mechanisms

Objective of Selection

- **Purpose:** Selection determines which individuals (solutions) reproduce and contribute to the next generation.
- **Balancing Exploration and Exploitation:**
 - Selection methods maintain a balance between **exploitation** of the best solutions and **exploration** of diverse, potentially suboptimal, solutions.

Common Selection Methods

- **Roulette Wheel Selection:**
 - Chromosomes are selected in proportion to their fitness scores. Better solutions have a higher probability of being chosen.
 - **Advantage:** Simple, probabilistic approach to selecting fitter individuals.
- **Tournament Selection:**
 - A group of individuals is randomly selected, and the best-performing one is chosen for reproduction.
 - **Advantage:** Higher selection pressure, often faster convergence.
- **Rank-Based Selection:**
 - Solutions are ranked by fitness, and selection is based on rank rather than raw fitness.
 - **Advantage:** Prevents highly fit solutions from dominating early in the process.
- **Elitism:**
 - The top-performing solutions are carried over to the next generation without modification.
 - **Advantage:** Guarantees the best solutions are retained, speeding up convergence.

Crossover Operators

Purpose of Crossover

- **Purpose:** Crossover combines genetic material from two parent solutions to generate offspring. This allows exploration of new areas of the solution space by combining traits of the parents.

Types of Crossover Operators

- **Single-Point Crossover:**
 - A random crossover point is selected, and genetic material is exchanged between parents from that point onward.
 - **Benefit:** Simple, fast, and effective for binary encoding.
- **Multi-Point Crossover:**
 - Multiple crossover points are selected, producing more complex combinations of parent genes.
 - **Benefit:** Allows for more diverse offspring compared to single-point crossover.
- **Uniform Crossover:**
 - Each gene is randomly selected from either parent, providing fine-grained mixing of genes.
 - **Benefit:** Ensures more diverse offspring with better distribution of genetic material.
- **Arithmetic Crossover:**
 - Offspring genes are generated as a weighted average of parent genes, suitable for continuous problems with real-number encoding.
 - **Benefit:** Helps maintain smooth transitions between generations in real-number spaces.

Crossover Rate

- **Definition:** The proportion of the population subjected to crossover.
- **Trade-off:** A higher crossover rate encourages more exploration, while a lower rate focuses more on exploitation of the current solutions.

Mutation Operators

Purpose of Mutation

- **Purpose:** Mutation introduces randomness by making small changes to the offspring's genes, helping to maintain diversity in the population and preventing premature convergence to local optima.

Types of Mutation

- **Bit-Flip Mutation:**
 - In binary encoding, a randomly selected bit is flipped (0 to 1 or vice versa).
 - **Benefit:** Simple and effective for binary-encoded problems.
- **Gaussian Mutation:**
 - In real-number encoding, a small random value from a Gaussian distribution is added to the gene.
 - **Benefit:** Maintains smooth transitions between solutions in continuous spaces.
- **Swap Mutation:**
 - In permutation encoding, two genes are randomly swapped.
 - **Benefit:** Maintains the validity of solutions where order matters (e.g., TSP).

Mutation Rate

- **Definition:** The probability that a gene will undergo mutation.
- **Trade-off:** A high mutation rate increases diversity but may introduce too much randomness, while a low mutation rate may lead to premature convergence.

Termination Criteria

Common Termination Conditions

- **Maximum Number of Generations:** The algorithm stops after a pre-defined number of generations, regardless of the fitness of the population.
- **Fitness Threshold:** The algorithm terminates once the best solution reaches a specified fitness value.
- **No Improvement:** If there has been no significant improvement in the best fitness score over a given number of generations, the algorithm terminates.
- **Time Limit:** The algorithm is terminated after a fixed period of runtime.

Convergence

- **Definition:** Convergence occurs when the population becomes homogeneous, with little or no genetic diversity.
- **Problem:** Premature convergence can result in suboptimal solutions.
- **Solutions:** Increase the mutation rate, use diversity-preserving techniques, or restart the population when convergence is detected.

Initialization and Population Representation

Encoding Solutions as Chromosomes

- **Binary Encoding:**
 - Each solution is represented as a binary string (e.g., `010101`).
 - Common in problems where variables have discrete or boolean values.
 - Example: In a knapsack problem, binary encoding could represent whether an item is included (1) or excluded (0) from the knapsack.
- **Real Number Encoding:**
 - Chromosomes are represented as arrays of real numbers, which is particularly useful for continuous variables.
 - Example: In optimization problems like minimizing a function over real values, real-number encoding allows for finer-grained control over solution precision.
- **Permutation Encoding:**
 - Useful for problems involving ordering, such as the traveling salesman problem (TSP).
 - Example: A chromosome represents a specific sequence of cities to visit, with each gene corresponding to a city.

Population Size

- **Trade-off between Exploration and Exploitation:**
 - A larger population size promotes greater **exploration** of the search space, reducing the risk of premature convergence. However, it also increases computational cost.
 - A smaller population focuses more on **exploitation**, zooming in on promising areas, but might miss global optima.

Diversity in Initial Population

- **Importance of Diversity:**
 - Ensuring diversity in the initial population helps the algorithm explore different areas of the search space, avoiding premature convergence to local optima.
 - Random initialization or using heuristics to generate a diverse set of initial solutions helps maintain variety in solutions.

function GENETIC-ALGORITHM(*population, fitness*) **returns** an individual
 repeat
 $weights \leftarrow \text{WEIGHTED-BY}(population, fitness)$
 $population2 \leftarrow \text{empty list}$
 for $i = 1$ **to** SIZE(*population*) **do**
 $parent1, parent2 \leftarrow \text{WEIGHTED-RANDOM-CHOICES}(population, weights, 2)$
 $child \leftarrow \text{REPRODUCE}(parent1, parent2)$
 if (small random probability) **then** $child \leftarrow \text{MUTATE}(child)$
 add *child* to *population2*
 $population \leftarrow population2$
 until some individual is fit enough, or enough time has elapsed
 return the best individual in *population*, according to *fitness*

function REPRODUCE(*parent1, parent2*) **returns** an individual
 $n \leftarrow \text{LENGTH}(parent1)$
 $c \leftarrow \text{random number from } 1 \text{ to } n$
 return APPEND(SUBSTRING(*parent1*, 1, c), SUBSTRING(*parent2*, $c + 1$, n))

Applications of Genetic Algorithms

Real-World Applications:

- **Traveling Salesman Problem (TSP):** Finding the shortest route to visit multiple cities.
- **Scheduling Problems:** Optimizing job schedules, task allocations.
- **Evolutionary Art and Design:** GA-generated art, evolving designs, or architectural layouts.
- **Engineering Design:** Structural optimization, circuit design.
- **Financial Market Prediction:** Optimizing trading strategies.
- **Machine Learning:** Feature selection, hyperparameter optimization.

Challenges and Limitations of Genetic Algorithms

- 1. Premature Convergence:** GAs can sometimes converge too quickly to a suboptimal solution, especially when the population loses diversity early in the search process. This occurs when most individuals in the population become similar, reducing the algorithm's ability to explore new areas of the solution space.
- 2. High Computational Cost:** GAs require evaluating a large number of potential solutions (chromosomes) across multiple generations. For problems with complex fitness functions or large solution spaces, this can result in high computational costs, making the algorithm slow or impractical for real-time applications.
- 3. Parameter Sensitivity:** The performance of GAs is highly sensitive to the choice of parameters, including population size, crossover rate, mutation rate, and selection pressure. Poor parameter settings can lead to inefficient search or failure to find a good solution.
- 4. Lack of Global Guarantees:** GAs do not guarantee finding the global optimal solution, especially in highly complex or noisy search spaces. Since GAs rely on stochastic processes (crossover, mutation), the final solution may vary from one run to another.
- 5. Slow Convergence for Some Problems:** GAs can be slow to converge to high-quality solutions, especially in problems with large search spaces or when the fitness landscape is flat (many solutions have similar fitness values).

6. Difficulty in Handling Constraints: Many optimization problems involve constraints, such as resource limits, physical boundaries, or logical conditions. GAs, by default, are not well-suited for handling constraints, which can lead to invalid solutions.

7. Poor Performance on Small or Simple Problems: For problems with small search spaces or simple objective functions, GAs may be overkill and can perform worse than simpler, deterministic algorithms like gradient descent or exhaustive search.

8. Fitness Function Design Challenges: Designing an appropriate fitness function is crucial, but it can be challenging, especially in multi-objective or complex problems. Poorly designed fitness functions can mislead the search, causing the GA to converge on suboptimal solutions.

Advanced Topics in Genetic Algorithms

- **Niching and Speciation:**
 - Techniques to maintain population diversity by forming sub-populations.
 - Crowding and Fitness Sharing to avoid dominance of a single species.
- **Hybrid Genetic Algorithms:**
 - Combining GAs with other optimization techniques like local search, hill climbing, or simulated annealing.
- **Multi-Objective Optimization:**
 - Problems with multiple objectives (e.g., minimize cost and maximize performance).
 - Pareto Front: A set of non-dominated solutions representing the trade-offs.
- **Parallel Genetic Algorithms:**
 - Implementing GAs in parallel to improve efficiency, especially for large search spaces.
 - Island models where multiple sub-populations evolve independently, with occasional migration of individuals.

N Queens using Genetic Algorithm

Step 1: Representation of Chromosomes (Encoding)

- **Chromosome:** A chromosome is a potential solution to the N-Queens problem. Each **chromosome** represents the **positions of queens** on an $N \times N$ chessboard.
- **Encoding:**
 - **Permutation encoding** is used: Each gene in the chromosome represents a **row**, and the value at each gene represents the **column** where the queen is placed.
 - Example for $N=8$:

Chromosome=[3,7,1,4,6,8,2,5]

- This means the queens are placed at:
- Column 1, Row 3
- Column 2, Row 7
- Column 3, Row 1
- Column 4, Row 4
- Column 5, Row 6
- Column 6, Row 8
- Column 7, Row 2
- Column 8, Row 5.

Step 2: Initial Population

- **Create an initial population** of chromosomes (potential solutions). The population size is typically a parameter to the algorithm (e.g., 50 or 100 chromosomes).
- **Random initialization:** Generate random valid positions of queens (i.e., ensuring that each queen is in a different column and different row) for each chromosome.

Step 3: Fitness Function

- The **fitness function** evaluates how good each chromosome (solution) is. For the N-Queens problem, the goal is to **minimize conflicts** between queens.
- **Fitness Calculation:**
 - The fitness of a chromosome is the **number of non-attacking pairs** of queens. The maximum number of non-attacking pairs for N-queens is:

$$\text{Max Non-Attacking Pairs} = \frac{N \times (N - 1)}{2}$$

- The fitness is calculated by subtracting the number of attacking pairs from this maximum value.

- **Attack Calculation:**
 - Queens are attacking each other if they are:
 - On the same **row**: This is prevented in the encoding.
 - On the same **diagonal**: Calculate the diagonals to see if any queens share the same diagonal.
- **Fitness Example:**
 - If $N=8$, the max non-attacking pairs = 28.
 - If a chromosome has 4 attacking pairs, the fitness is: **Fitness=28-4=24**

Step 4: Selection

- **Select parents** for reproduction (crossover). Use a **selection mechanism** to pick chromosomes with a higher fitness to produce the next generation.
- Common selection methods include:
 - **Roulette Wheel Selection**: The probability of selecting a chromosome is proportional to its fitness.
 - **Tournament Selection**: Randomly pick a few chromosomes, and select the best one out of the group.
- **Example:**
 - Assume we have 5 chromosomes with fitness values 20, 22, 24, 26, and 28. Chromosome with fitness 28 will have the highest probability of being selected for reproduction.

Step 5: Crossover (Recombination)

- Perform **crossover** to generate new chromosomes (offspring) by combining genes from two parent chromosomes.
- **Crossover Method:**
 - **One-point crossover** is common for the N-Queens problem:
 - Randomly select a crossover point in the chromosome, and swap the sections between two parents.
- **Example:**
 - Parent 1: [3,7,1,4,6,8,2,5]
 - Parent 2: [4,1,5,2,7,3,8,6]
 - Random crossover point: 4 (meaning after 4th gene)
 - Offspring:
 - Offspring 1: [3,7,1,4,7,3,8,6]
 - Offspring 2: [4,1,5,2,6,8,2,5]
 - **Note:** This might result in invalid chromosomes (duplicate positions), so after crossover, you may apply a **repair function** to ensure no duplicates.

Step 6: Mutation

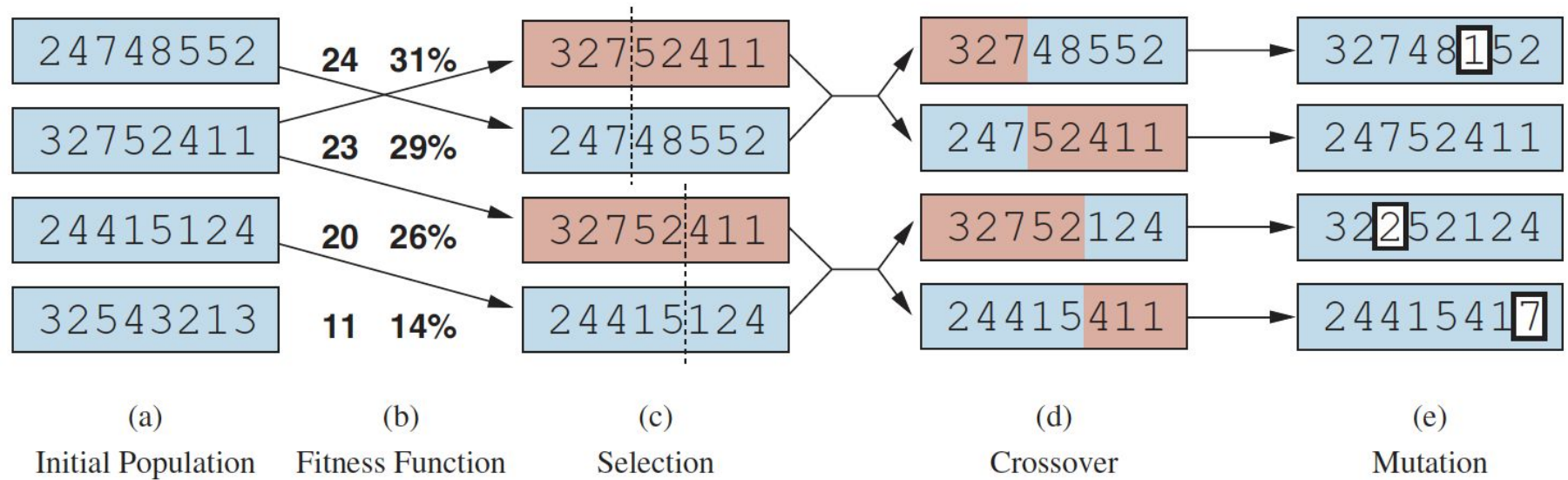
- **Mutation** introduces randomness and helps maintain genetic diversity in the population, preventing the algorithm from getting stuck in local optima.
- **Mutation Method:**
 - For N-Queens, a typical mutation is to **swap two random genes** (swap the positions of two queens).
- **Example:**
 - Chromosome before mutation: [3,7,1,4,6,8,2,5]
 - After mutation: [3,7,1,6,4,8,2,5] (Here, queens in positions 5 and 4 are swapped).
- **Mutation Rate:**
 - Mutation usually happens with a low probability (e.g., 1% or 5%).

Step 7: Replacement

- After generating new offspring through crossover and mutation, **replace** the old population with the new population.
- **Replacement Strategy:**
 - The new population may consist entirely of the offspring, or you can use **elitism**, where the best-performing individuals from the current generation are carried forward to the next generation to preserve their good traits.

Step 8: Termination Condition

- The algorithm continues for several generations, applying selection, crossover, and mutation in each generation.
- **Termination Criteria:**
 - The algorithm stops when one of the following occurs:
 1. A chromosome with a **fitness of 28** (for $N=8$) is found, meaning a valid solution has been identified (no conflicts).
 2. A **maximum number of generations** is reached (e.g., 1000 generations).
 3. The **population has converged** (no significant improvement over several generations).



A genetic algorithm, illustrated for digit strings representing 8-queens states. The initial population in (a) is ranked by a fitness function in (b) resulting in pairs for mating in (c). They produce offspring in (d), which are subject to mutation in (e).

Prompt-Based Editing for Text Style Transfer

3.2 Search Objective

We apply an edit-based search for unsupervised style transfer. This follows the recent development

In our work, we use the SAHC algorithm: in a search step t , SAHC enumerates every editing position and performs every editing operation (namely, word deletion, replacement, and insertion)⁵. Then it selects the highest-scored candidate sentence $\mathbf{y}^*$ if the score $f(\mathbf{y}^*, \mathbf{x})$ is higher than $f(\mathbf{y}^{(t-1)}, \mathbf{x})$ before it reaches the maximum edit steps. Otherwise, SAHC terminates and takes the candidate $\mathbf{y}^{(t-1)}$ as the style-transferred output. In this way, our SAHC greedily finds the best edit for every search step and is more powerful than SA and FCHC in our

3.3 Discrete Search Algorithm

We perform style-transfer generation by discrete local search using editing operations, such as word insertion, deletion, and replacement, following pre-

Dataset	Algorithm	ACC%	BLEU	GM	HM
YELP	SAHC	73.0	40.1	54.1	51.7
	FCHC	67.2	31.8	46.2	43.1
	SA	66.0	28.7	43.5	40.0
AMAZON	SAHC	72.7	28.6	45.6	41.0
	FCHC	64.1	24.8	39.8	35.7
	SA	63.2	23.7	38.7	34.4

Table 5: Results of different search algorithms on the sentiment transfer datasets.

Unsupervised Paraphrasing by Simulated Annealing

We propose UPSA, a novel approach that accomplishes Unsupervised Paraphrasing by Simulated Annealing. We model paraphrase generation as an optimization problem and propose a sophisticated objective function, involving semantic similarity, expression diversity, and language fluency of paraphrases. UPSA searches the sentence space towards this objective by performing a sequence of local edits.

Line #	UPSA Variant	iBLEU	BLEU	Rouge1	Rouge2
1	UPSA	12.41	18.48	57.06	31.39
2	w/o $f_{\text{sim,key}}$	10.28	15.34	50.85	26.42
3	w/o $f_{\text{sim,sen}}$	11.78	17.95	57.04	30.80
4	w/o f_{exp}	11.93	21.17	59.75	34.91
5	w/o copy	11.42	17.25	56.09	29.73
6	w/o annealing	10.56	16.52	56.02	29.25

Let $\mathcal{X}$ be a (huge) search space of sentences, and $f(x)$ be an objective function. The goal is to search for a sentence x that **maximizes** $f(x)$. At a searching step t , SA keeps a current sentence x_t , and proposes a new candidate x_* by local editing. If the new candidate is better scored by f , i.e., $f(x_*) > f(x_t)$, then SA accepts the proposal. Otherwise, SA tends to reject the proposal x_* , but may still accept it with a small probability $e^{\frac{f(x_*) - f(x_t)}{T}}$, controlled by an annealing temperature T . In other words, the probability of accepting the proposal is

$$p(\text{accept}|x_*, x_t, T) = \min \left(1, e^{\frac{f(x_*) - f(x_t)}{T}} \right). \quad (1)$$

Simulated Annealing for Emotional Dialogue Systems

ions. In this study, we consider the task of expressing a specific emotion for dialogue generation. Previous approaches take the emotion as an input signal, which may be ignored during inference. We instead propose a search-based emotional dialogue system by simulated annealing (SA). Specifically, we first define a scoring function that combines contextual coherence and emotional correctness. Then, SA iteratively edits a general response and searches for a sentence with a higher score, enforcing the presence of the desired emotion. We evaluate our system on the NLPCC2017 dataset.

2.3 Search by Simulated Annealing (SA)

We use the simulated annealing (SA) algorithm to search for a desired utterance. SA starts from a general dialogue response $\mathbf{y}^{(0)} = \operatorname{argmax} P_{\text{Seq2Seq}}(\mathbf{y}|\mathbf{x})$, obtained by standard beam search (BS) or diverse beam search (DBS) on the trained Seq2Seq model. Then, SA maximizes the scoring function (5) by iteratively editing the candidate response.

Models		BLEU Scores		Diversity		Embedding-Based Metrics				Emotion accuracy
		BLEU-1	BLEU-2	Dist-1	Dist-2	Average	Greedy	Extreme	Coherence	
Previous	Seq2Seq	4.24	0.73	0.035	0.119	0.497	0.328	0.352	0.582	0.244
	EmoEmb	7.22	1.64	0.040	0.133	0.532	0.356	0.381	0.594	0.693
	EmoDS	9.76	2.82	0.050	0.174	0.623	0.403	0.427	0.603	0.746
	ECM	10.23	3.32	0.052	0.177	0.625	0.405	0.433	0.607	0.753
	CDL	12.54	3.70	0.065	0.221	0.642	0.438	0.457	0.635	0.823
Ours	Seq2Seq BS	10.78	3.11	0.058	0.215	0.765	0.543	0.594	0.690	0.253
	Seq2Seq BS + SA	13.90	4.03	0.051	0.276	0.782	0.569	0.610	0.701	0.928
	Seq2Seq DBS	12.14	3.89	0.061	0.209	0.768	0.545	0.601	0.699	0.264
	Seq2Seq DBS + SA	14.26	4.12	0.053	0.239	0.786	0.556	0.611	0.703	0.942